AfDB – DI4A

HANDS-ON SESSION

Query your institution's documents

Retrieval-Augmented Generation (RAG) for statistical publications: grounded, cited, and able to say “I don't know”



October 2026

AfDB – Data Innovation for Africa (DI4A)

THIS SESSION · 30 MINUTES

Four objectives, thirty minutes

01

Build an assistant for statistical documents

Load your institution's PDFs and see what was extracted.

Notebook A1–A2 · 7 min

02

Practise search & sourced answers

Ask, then check document, section, page and figures.

A3 · C1 · 6 min

03

Compare retrieval methods

Keywords, meaning, hybrid, reranking; five ways to cut a report.

B3 · B4 · 5 min

04

Diagnose errors and out-of-scope questions

Threshold, refusals, figure checks, a first test set.

B5 · B8 · B9 · 5 min

Concepts · 7 min	Hands-on in the notebook · 21 min	2′

Slides carry the concepts; the notebook does the rest. The in-depth track (90 min) is for further practice.

Why an LLM alone is not enough

An LLM is a brilliant reasoner…

…but an unreliable library.

Its knowledge is stored implicitly in its weights (parametric memory): frozen, opaque and impossible to cite.

Knowledge cutoff

Anything that happened after training is unknown to the model.

Hallucinations

When knowledge is missing, the model can produce fluent but unsupported answers.

No private data

Internal documents, wikis, tickets and databases were never in the training set.

No traceability

Answers come without sources, so users cannot verify them.

What is Retrieval-Augmented Generation?

RAG retrieves relevant passages from an external knowledge source at query time and injects them into the prompt, so the LLM generates an answer **grounded in that evidence.**



Closed-book exam

LLM alone

Answers from memory only (model weights). May be outdated or invented.

Parametric memory only



Open-book exam

LLM + RAG

Looks up the right pages first, then answers — and can cite them.

Parametric + non-parametric memory

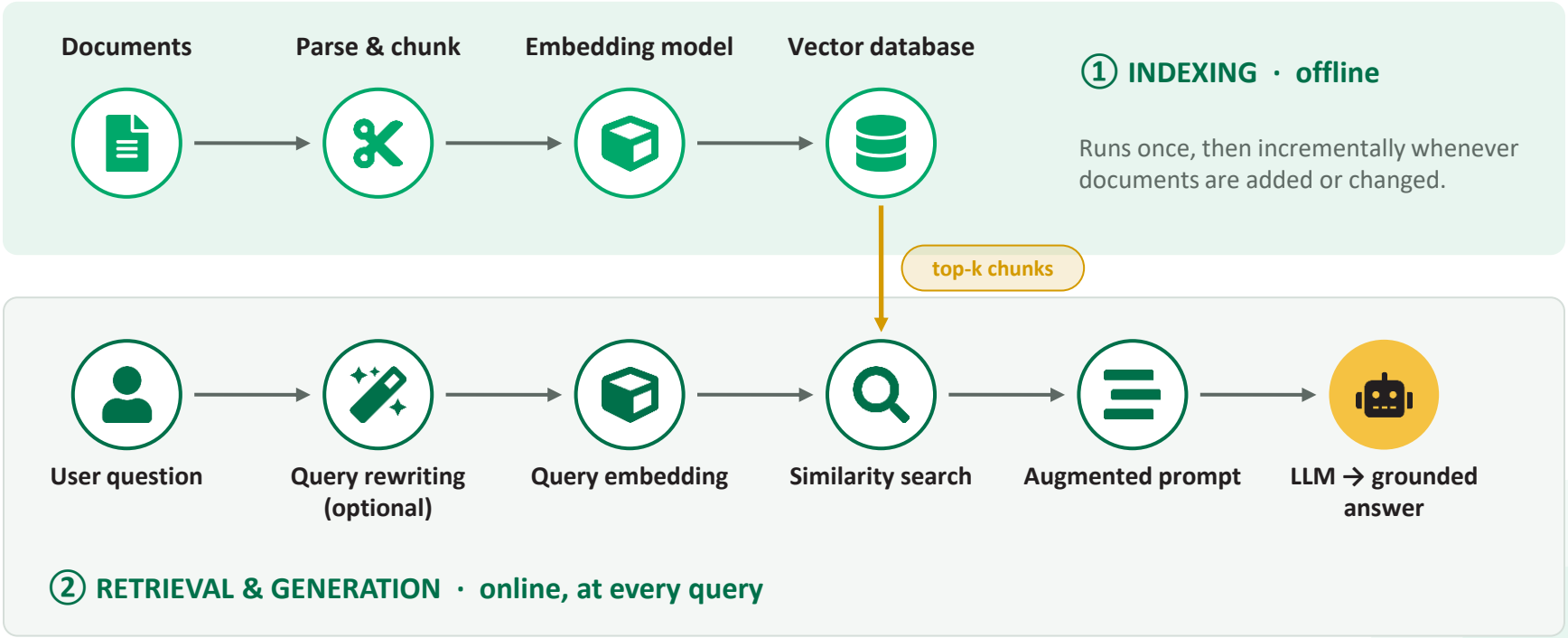
ORIGIN

Lewis et al., 2020

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — NeurIPS 2020 (Facebook AI Research, UCL, NYU).

Combines parametric memory (BART) with non-parametric memory (a DPR index over Wikipedia).

Two pipelines: indexing and querying



RAG in action: one question, end to end

1 Question

“What was the unemployment rate in the second quarter of 2025?”

2 Embed

`[ 0.021, −0.487, 0.913, 0.106, … ]` *same model as the documents*

3 Retrieve & rerank

`0.91` [1] Labour Market Report 2025, Table 1 — unemployment rate, Q2: 7.8%
`0.78` [2] Labour Market Report 2025, §2 Key findings — annual average 7.9%
`0.35` Census 2024 — population by region *(dropped)*

4 Augment

Instructions + chunks [1] [2] + question → one prompt sent to the LLM

5 Generate

“The unemployment rate stood at 7.8% in the second quarter of 2025 [1]; the annual average was 7.9% [2].”

Illustrative — the fictitious reports of the notebook; vectors and scores simplified.

Sparse, dense, hybrid — then rerank

Sparse · BM25

Keyword matching with term-frequency weighting.

Best for exact terms, product codes, names, rare words.

Dense · Embeddings

Vector similarity on meaning.

Best for paraphrases, synonyms, natural-language questions.

Hybrid

Runs both and fuses the ranked lists, e.g. with Reciprocal Rank Fusion.

$$RRF(d) = \Sigma\ 1 / (k + rank_i(d))$$

RERANKING

Two-stage retrieval: fast recall first, precise scoring second.

Top-50 / 100 candidates
Bi-encoder or BM25 · fast

→

Cross-encoder rescoring
Reads query + passage together · accurate

→

Top-3 to 10
Sent to the LLM

Building the augmented prompt

SYSTEM

```
Answer ONLY from the sources below, copying
figures with their unit and period. Cite [n].
Otherwise write: "Information not available in
the documents."
```

CONTEXT

```
[1] Labour Market Report 2025, Table 1 —
Unemployment rate — Q2: 7.8%…
[2] Labour Market Report 2025, §2 — Annual
average unemployment rate: 7.9%…
```

QUESTION

```
What was the unemployment rate in the second
quarter of 2025?
```

Simplified from the prompt used in the notebook

Require citations

Each claim is linked to a retrieved chunk the user can check.

Allow abstention

"I don't know" is better than a confident hallucination.

Order matters

Place the strongest evidence first: models under-use the middle of long contexts (see Evidence).

Keep context focused

Fewer, more relevant chunks usually beat many noisy ones.

HANDS-ON · SET-UP

Get started in three minutes

1

Open in Colab

Open the notebook, sign in with Google, then Runtime → Run all.

2

Choose the model

Fastest in the room: a free Groq key (PROVIDER = "groq", step 0b). Ollama keeps everything inside the session.

3

Load a PDF

Default: the AfDB – DI4A guide. Then a publication of yours: UPLOAD_PDF = True.

4

Run the Express track

A1 load · A2 prepare · A3 ask. No model? Extractive mode still quotes the right sentence.

Confidentiality: published documents only with an online model (Groq, OpenAI, Anthropic). Unpublished or embargoed material stays with Ollama — and no public Gradio link without WEB_PASSWORD.

HANDS-ON · EXERCISES

Four exercises, one per objective

1 · Build your assistant

NOTEBOOK A1–A2 · ≈ 7 MIN

Run on the AfDB guide, then on a publication of your institution (CPI note, QNA release, BoP bulletin, policy statement). Read pages, tables, scanned pages, threshold.

2 · Ask and verify

NOTEBOOK A3 · C1 · ≈ 6 MIN

Three questions: a figure in a table, a methodological question, a follow-up. Check source, page thumbnail, figure check and “Understood as…”.

3 · Compare retrieval methods

NOTEBOOK B3–B4 · ≈ 5 MIN

compare_chunking(), then benchmark() on 3–5 questions you can answer. explain() shows what BM25, semantic search and reranking each found.

4 · Diagnose and decline

NOTEBOOK B5 · B8 · B9 · ≈ 5 MIN

Two out-of-scope questions: it must decline. Trace a wrong answer with explain(), then write a four-question test set.

The gold boxes in the notebook are these four exercises. Your notes from 2 and 4 become your test set.

Failure points and best practices

7 failure points

Barnett et al., 2024

1. Missing content — the answer is not in the corpus
2. Missed top-ranked documents
3. Not in context — dropped during consolidation
4. Not extracted — present but ignored by the LLM
5. Wrong format
6. Incorrect specificity
7. Incomplete answer

Best practices

What consistently works

- ✓ Build an evaluation set before optimising
- ✓ Clean parsing & structure-aware chunking
- ✓ Hybrid search + reranking as a default
- ✓ Metadata filters and permission checks
- ✓ Grounding prompt with citations & abstention
- ✓ Monitor queries, refresh the index, iterate

From the notebook to your own project

Methodology helpdesk

CPI or national-accounts manuals and procedures, for compilers and new staff.

Release Q&A

Releases, release calendar and revision policy, for users and the press.

BoP classification support

BPM6 chapters and code lists — complements machine-learning classification of transactions.

Monetary policy assistant

Statements, minutes and inflation reports, for economists and researchers.

Bilingual assistant

The same reports in English and French; ask in either (multilingual notebook).

What your team records

Users and problem · documents and confidentiality · 10–20 test questions incl. refusals · benchmark · next step.

AfDB – Data Innovation for Africa (DI4A) can help take a prototype further.

KEY TAKEAWAYS

Retrieval quality is the ceiling of answer quality.



Grounded

RAG gives LLMs current, private and citable knowledge without retraining.



Retrieval first

Chunking, hybrid search and reranking drive most of the gains.



Measured

Evaluate retrieval and generation separately, on your own data.



Evolving

From fixed pipelines to adaptive, agentic retrieval that decides when and what to search.

Start simple · measure on your own questions · take it to your own project.

APPENDIX

A

Reference material

The full RAG briefing for self-study: foundations, evidence, evolution, techniques, production



The full briefing: six parts, from concept to production

01

Foundations

Why LLMs need retrieval, what RAG is, where it is used

02

How it works

Indexing, chunking, embeddings, retrieval and generation

03

Evidence

Published results that quantify the gains

04

Evolution of RAG

Generations, dimensions of progress, paradigms

05

Advanced techniques

Toolbox; RAG vs fine-tuning vs long context

06

Production

Evaluation, security, tooling, failure points

SECTION

01

Foundations

Why do language models need an external memory?



Where RAG creates value

Customer support

Assistants answer from help-center articles, policies and past tickets, with links to sources.

Enterprise knowledge

One question-answering layer over wikis, intranets, drives and chat history.

Legal & compliance

Search contracts, regulations and case files; every statement traceable to a clause.

Healthcare & research

Ground answers in guidelines and scientific literature instead of model memory.

Software engineering

Code assistants retrieve relevant files, APIs and internal documentation.

Finance & analysis

Question reports, filings and market research that change every quarter.

Common pattern: a large, changing or private corpus + a need for answers users can verify.

SECTION

02

How it works

What happens between a question and a grounded answer?



Turning documents into searchable knowledge

01

Load & parse

Extract clean text from PDFs, HTML, Office files, wikis, databases. Keep tables and headings intact.

02

Chunk

Split text into passages sized for retrieval, typically a few hundred tokens, often with overlap.

03

Embed

An embedding model maps each chunk to a dense vector that captures its meaning.

04

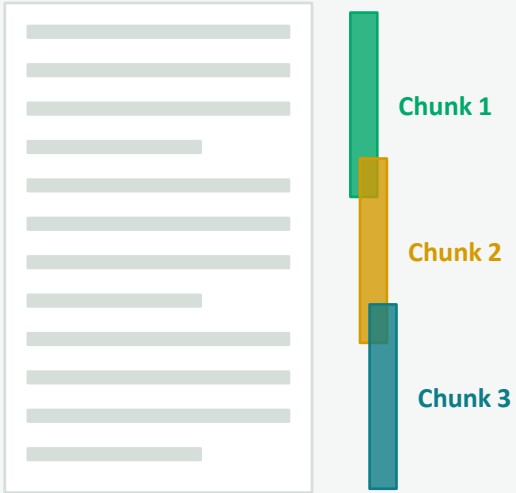
Store & index

Save vectors + text + metadata (source, date, access rights) in a vector index.

Key rule: the same embedding model must be used for documents and for queries — otherwise the vectors are not comparable.

The most underestimated design decision

Fixed-size chunks with overlap



Overlapping zones keep sentences that cross a boundary retrievable.

Common strategies

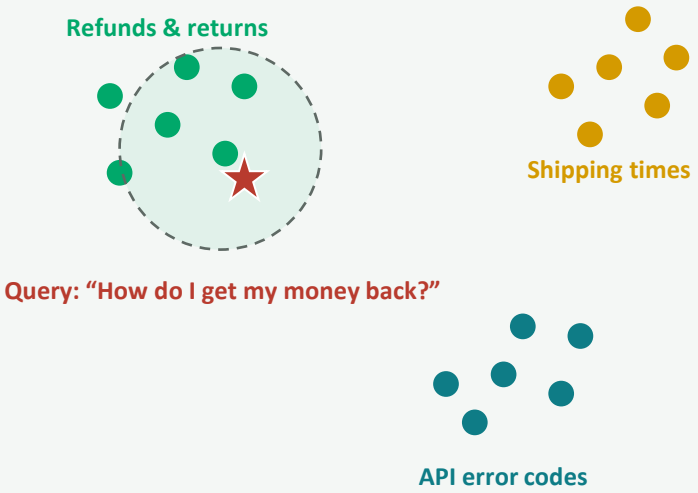
1. **Fixed-size + overlap**
   Simple and predictable; may cut ideas mid-sentence.
2. **Recursive splitting**
   Splits on paragraphs, then sentences, then words until the size fits.
3. **Semantic chunking**
   Starts a new chunk where the embedding similarity between sentences drops.
4. **Structure-aware**
   Follows headings, sections, tables, code blocks or Markdown structure.

Too small
Precise match, but context is lost.

Too large
Rich context, but diluted, noisy vectors.

Embeddings: searching by meaning, not by words

Embedding space (2-D projection, illustrative)



No shared keyword with “refund”, yet its vector lands next to refund passages.

Similarity measure

$$\cos(q, d) = (q \cdot d) / (\|q\| \, \|d\|)$$

Score close to 1 = same meaning. Dot product and Euclidean distance are also common.

Approximate Nearest Neighbour indexes

HNSW	Multi-layer proximity graph; fast, high recall
IVF	Clusters vectors; searches only nearby clusters
PQ	Compresses vectors to cut memory usage

ANN trades a tiny loss of exactness for millisecond search over millions of vectors.

SECTION

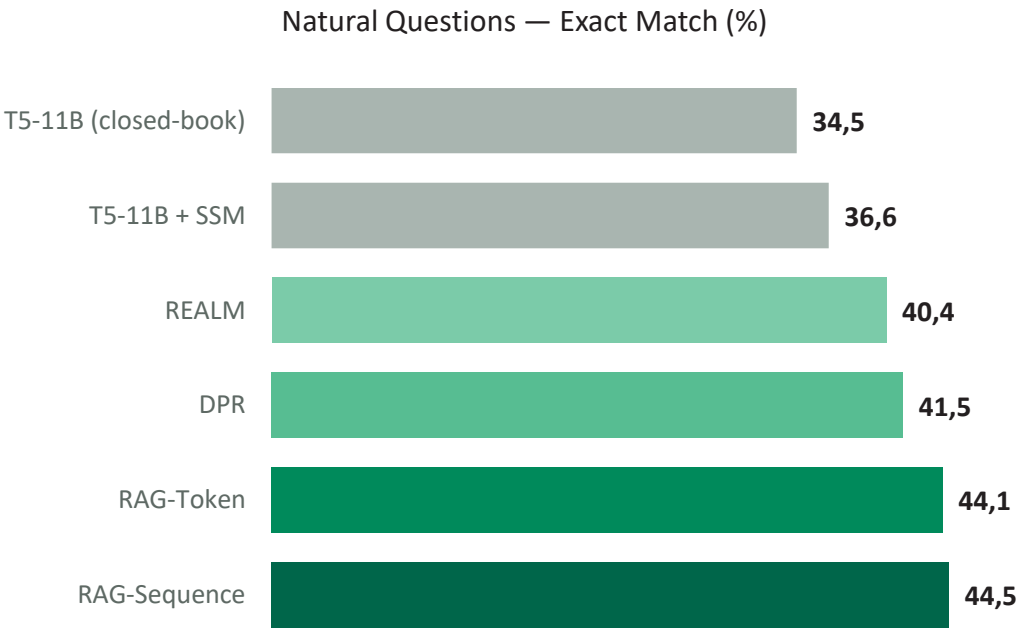
03

Evidence

Does retrieval really improve answers?



Retrieval beats sheer model size



626M

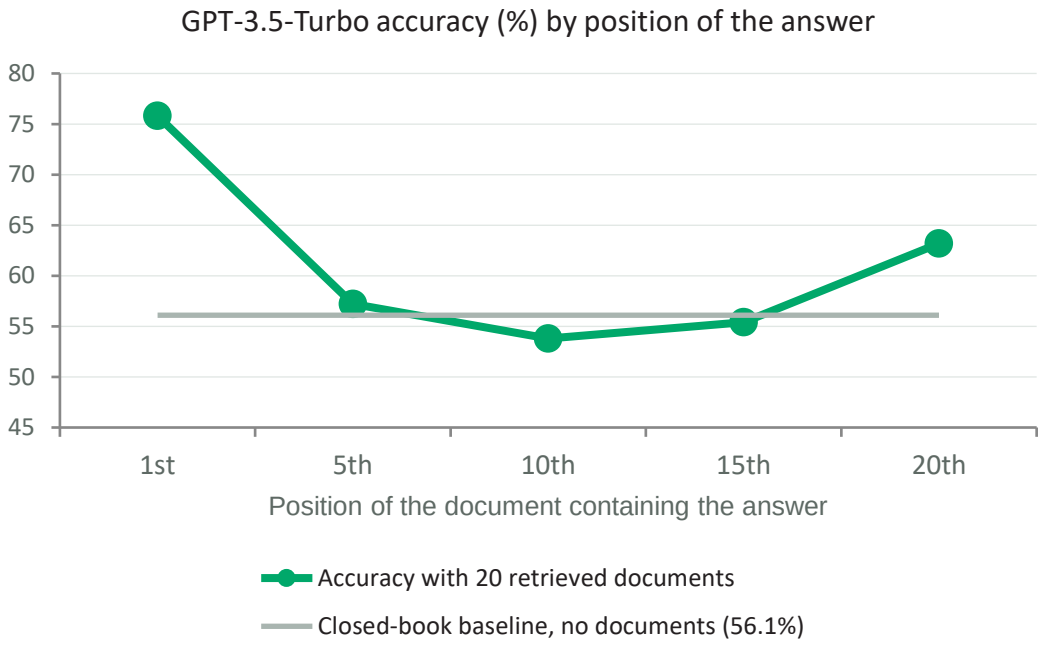
trainable parameters for RAG — vs 11B for T5-11B, which it outperforms.

42.7% vs 7.1%

Jeopardy generation: share of human judgments rating RAG vs BART as more factual.

Source: Lewis et al., NeurIPS 2020 (Tables 1 & 4). SSM = salient span masking pre-training.

Lost in the Middle: position matters



Source: Liu et al., "Lost in the Middle", TACL 2024 (Table 6). Multi-document QA on NaturalQuestions-Open.

–22 pts

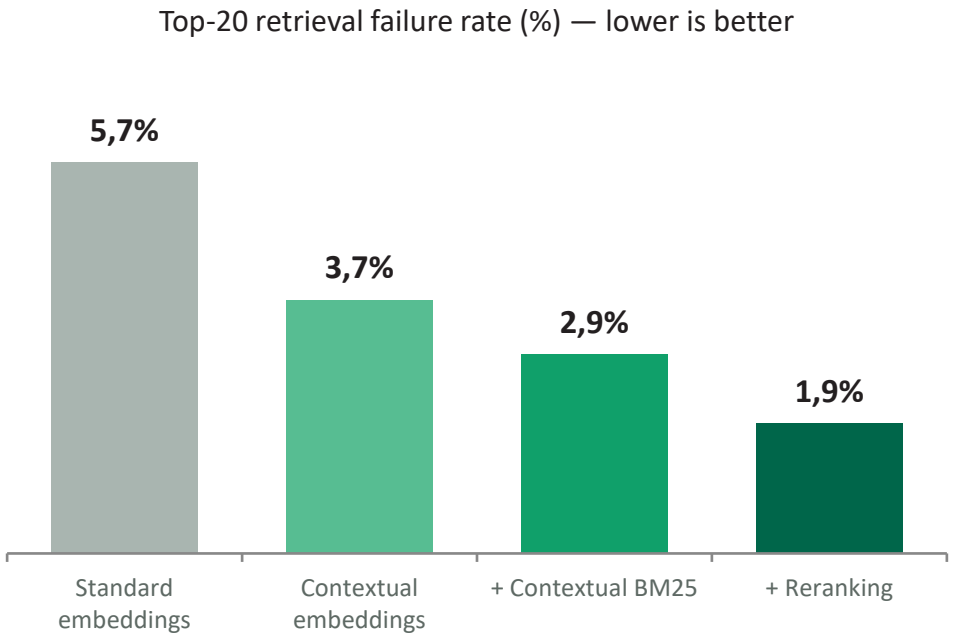
from 75.8% (answer first) to 53.8% (answer in the middle).

Below baseline

Mid-context, accuracy is worse than giving the model no documents at all (56.1%).

For RAG: rerank so the best evidence comes first, and send fewer, better chunks.

Contextual Retrieval: restoring lost context



Source: Anthropic, "Introducing Contextual Retrieval", September 2024.

The problem

Once split, a chunk loses who and when it refers to:

Before: "Revenue grew by 3% over the previous quarter."

After: "[ACME Corp, Q2 filing] Revenue grew by 3%…"

The fix

An LLM writes a short context (≈50–100 tokens) situating each chunk within its document, prepended before embedding and BM25 indexing.

–49% failures · **–67%** with reranking

SECTION

04

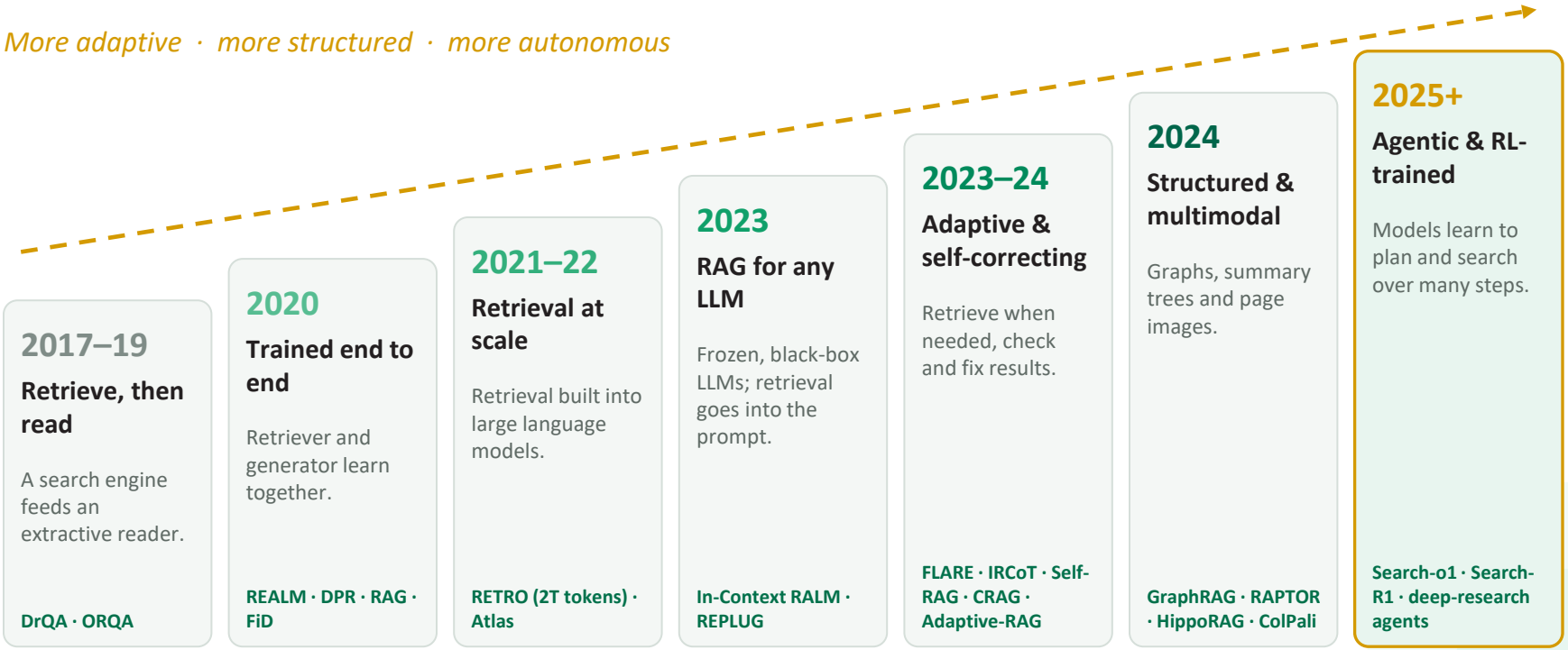
Evolution of RAG

How has the idea of RAG matured since 2017?



How the RAG concept has progressed

More adaptive · more structured · more autonomous



Five dimensions of progress

Dimension	Early RAG (2020)	Established practice	Frontier
When to retrieve	Once, before generating	Several times, interleaved with reasoning (IRCoT, FLARE)	**The model decides if and when (Self-RAG, Adaptive-RAG)**
What is retrieved	Fixed-size text passages	Hybrid, reranked, contextualised chunks	**Knowledge graphs, summary trees, page images (GraphRAG, RAPTOR, ColPali)**
How search is learned	Retriever trained jointly with a small generator	Frozen LLM; retriever off-the-shelf or tuned from LLM feedback (REPLUG)	**LLM trained by reinforcement learning to search (Search-R1)**
Who controls the flow	Fixed linear pipeline	Modular pipelines with routing	**Agents that plan multi-step research**
Quality control	Ranking only	Reranking, citations, automated evaluation (RAGAS)	**Self-critique and corrective retrieval (Self-RAG, CRAG)**

Direction of progress →

From Naive to Advanced to Modular RAG

Naive RAG

Index → Retrieve → Generate, in a fixed linear chain.

- Low retrieval precision
- Redundant or noisy context
- Hallucinations remain

Advanced RAG

Adds optimisations before and after retrieval.

- Pre: query rewriting & expansion, better indexing
- Post: reranking, context compression
- Hybrid search, metadata filters

Modular RAG

Interchangeable modules and flexible flows.

- Routing, search, memory, fusion modules
- Iterative & adaptive retrieval
- Agent-driven orchestration

Additive, not a replacement: each paradigm builds on the components of the previous one.

Framework from Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey", 2023.

From a one-shot pipeline to a reasoning loop

BEFORE · ONE SHOT

- Question
- Retrieve once
- Generate

If the first search misses, the answer fails.

NOW · AGENTIC LOOP

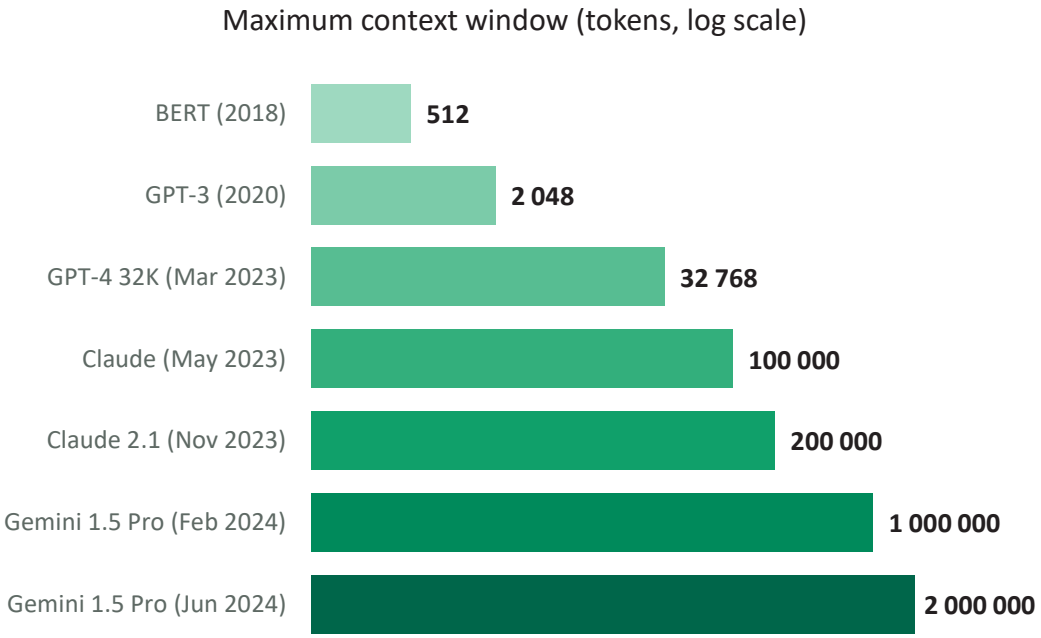


IRCoT *Trivedi et al., 2023*
Interleaves retrieval with each chain-of-thought step.

FLARE *Jiang et al., 2023*
Retrieves when the next sentence contains low-confidence tokens.

Search-R1 *Jin et al., 2025*
LLM trained with RL to issue multiple searches while reasoning.

Bigger context windows changed RAG's role



Figures as announced by each provider.

Then: RAG was a workaround — models could read only a few pages at once.

Evidence: long-context models can beat RAG on average, but RAG costs far less. Self-Route (Li et al., 2024) tries RAG first and falls back to full context only when needed.

Now: RAG is the layer for cost, freshness, access control and precision.

SECTION

05

Advanced techniques

Which techniques push answer quality further?



Advanced RAG techniques in practice

Gao et al., 2022

HyDE

LLM writes a hypothetical answer; its embedding is used to search.

Query transformation

Multi-query rewriting

Rewrites or splits a question into sub-queries, then merges results.

Parent-document

Small-to-big retrieval

Matches on small chunks, returns the larger parent section.

Asai et al., 2023

Self-RAG

Model decides when to retrieve and critiques outputs via reflection tokens.

Yan et al., 2024

Corrective RAG (CRAG)

An evaluator grades retrieved docs and triggers corrective actions such as web search.

Microsoft, 2024

GraphRAG

Builds a knowledge graph + community summaries to answer global questions.

Sarthi et al., 2024

RAPTOR

Recursive tree of clustered summaries for multi-level retrieval.

Agents + tools

Agentic RAG

An agent plans, calls retrieval tools iteratively and verifies before answering.

Query & indexing Research methods Agentic

RAG vs fine-tuning vs long context

	RAG	Fine-tuning	Long context
Updating knowledge	**Re-index documents**	Retrain the model	**Change the prompt**
Source citations	**Native (retrieved chunks)**	Not available	Possible
Knowledge volume	**Millions of documents**	Absorbed into weights, imprecise	Bounded by the context window
Cost per query	Low — only relevant chunks	**Low**	High for large contexts
Access control	**Filter by user rights at retrieval**	Very hard	Per request
Best for	**Private & changing facts**	**Style & specialised behaviour**	**Small, self-contained corpora**

Complementary, not exclusive: fine-tune for behaviour, RAG for knowledge, long context for small corpora.

SECTION

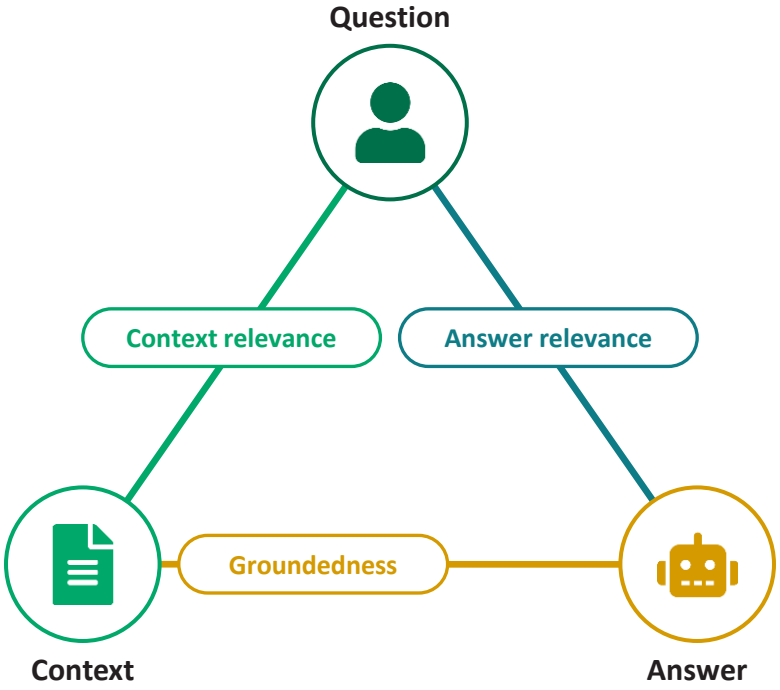
06

Production

How do you run RAG reliably and safely?



Measure retrieval and generation separately



The "RAG triad" (TruLens)

RAGAS metrics (Es et al., 2023)

Faithfulness: Are the answer's claims supported by the context?

Answer relevancy: Does the answer address the question?

Context precision: Are relevant chunks ranked at the top?

Context recall: Was all needed information retrieved?

Retrieval metrics

Recall@k · Precision@k · MRR · nDCG

Tools: RAGAS · TruLens · DeepEval · Arize Phoenix

Securing a RAG system

Indirect prompt injection

Hidden instructions inside ingested documents are read by the LLM as if they were commands.

Knowledge poisoning

PoisonedRAG (Zou et al., 2024): 5 crafted texts per target question reached ~90% attack success in a corpus of millions.

Permission leakage

Vectors do not inherit source-system permissions; users or tenants may retrieve what they should not see.

Embedding inversion

Research shows much of the original text can be reconstructed from its embedding (Morris et al., 2023).

Mitigations

- Validate and sanitise documents at ingestion
- Enforce access control at retrieval; partition tenants
- Treat retrieved text as data, never as instructions
- Restrict what tools and actions the LLM can trigger
- Encrypt vector stores; log and monitor retrievals

OWASP Top 10 for LLM Applications 2025 now lists “LLM08: Vector and Embedding Weaknesses”.

The RAG technology stack

Orchestration	Parsing	Embeddings	Vector stores	Rerankers
LangChain	Unstructured	OpenAI text-embedding-3	Pinecone · Weaviate	Cohere Rerank
LlamaIndex	Docling	Cohere Embed	Qdrant · Milvus	BGE-reranker
Haystack	LlamaParse	Voyage AI	Chroma · pgvector	ColBERT
DSPy	Apache Tika	BGE · E5 (open)	Elasticsearch · FAISS	Cross-encoders

Tip: compare embedding models on the MTEB benchmark, then validate on your own data and languages (e.g. French, Arabic).

References (1/2)

- Chen et al. (2017). Reading Wikipedia to Answer Open-Domain Questions (DrQA). ACL.
- Lee et al. (2019). Latent Retrieval for Weakly Supervised Open Domain QA (ORQA). ACL.
- Guu et al. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. ICML.
- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP.
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Izacard & Grave (2021). Leveraging Passage Retrieval with Generative Models (FiD). EACL.
- Borgeaud et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens (RETRO). ICML.
- Izacard et al. (2022). Atlas: Few-shot Learning with Retrieval Augmented Language Models.
- Gao, Ma et al. (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).
- Ram et al. (2023). In-Context Retrieval-Augmented Language Models. TACL.
- Shi et al. (2023). REPLUG: Retrieval-Augmented Black-Box Language Models.
- Trivedi et al. (2023). Interleaving Retrieval with Chain-of-Thought Reasoning (IRCoT). ACL.
- Jiang et al. (2023). Active Retrieval Augmented Generation (FLARE). EMNLP.
- Asai et al. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.
- Liu et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. TACL 2024.
- Gao, Xiong et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey.

References (2/2)

- Es et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation.
- Morris et al. (2023). Text Embeddings Reveal (Almost) As Much As Text. EMNLP.
- Yan et al. (2024). Corrective Retrieval Augmented Generation (CRAG).
- Jeong et al. (2024). Adaptive-RAG: Adapting Retrieval-Augmented LLMs through Question Complexity. NAACL.
- Edge et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft.
- Sarthi et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR.
- Gutiérrez et al. (2024). HippoRAG: Neurobiologically Inspired Long-Term Memory for LLMs. NeurIPS.
- Faysse et al. (2024). ColPali: Efficient Document Retrieval with Vision Language Models.
- Li et al. (2024). Retrieval Augmented Generation or Long-Context LLMs? (Self-Route). EMNLP Industry.
- Barnett et al. (2024). Seven Failure Points When Engineering a RAG System.
- Zou et al. (2024). PoisonedRAG: Knowledge Corruption Attacks to RAG of LLMs.
- Anthropic (2024). Introducing Contextual Retrieval.
- Li et al. (2025). Search-o1: Agentic Search-Enhanced Large Reasoning Models.
- Jin et al. (2025). Search-R1: Training LLMs to Reason and Leverage Search Engines with RL.
- OWASP (2025). Top 10 for LLM Applications — LLM08: Vector and Embedding Weaknesses.
- Cormack et al. (2009). Reciprocal Rank Fusion. SIGIR · Malkov & Yashunin (2018). HNSW. IEEE TPAMI.